JC3504 机器人技术讲座 2：机器人操作系统（ROS）

大纲

介绍与架构

ROS 核心概念

- 节点和包
- 主题和消息
- 节点的发现
- 服务
- 参数

与 ROS 一起踏上我们的旅程

开发你的第一个节点

机器人操作系统 (ROS)

机器人操作系统（ROS）是一个用于编写机器人软件的灵活框架，提供了一系列工具、库和约定，旨在简化创建强大且多功能机器人应用程序的复杂过程。

Robot Operating System (ROS)

<video 02 - ROS Intro.mp4>

History of ROS

History of ROS

2006年，Morgan Quigley是斯坦福大学的博士生，专攻机器人学，在Andrew Ng的指导下学习。

摩根·奎格利是斯坦福人工智能机器人（STAIR）项目的成员之一，负责机器人代码的整合和适配。

安德鲁·吴

摩根·奎格利

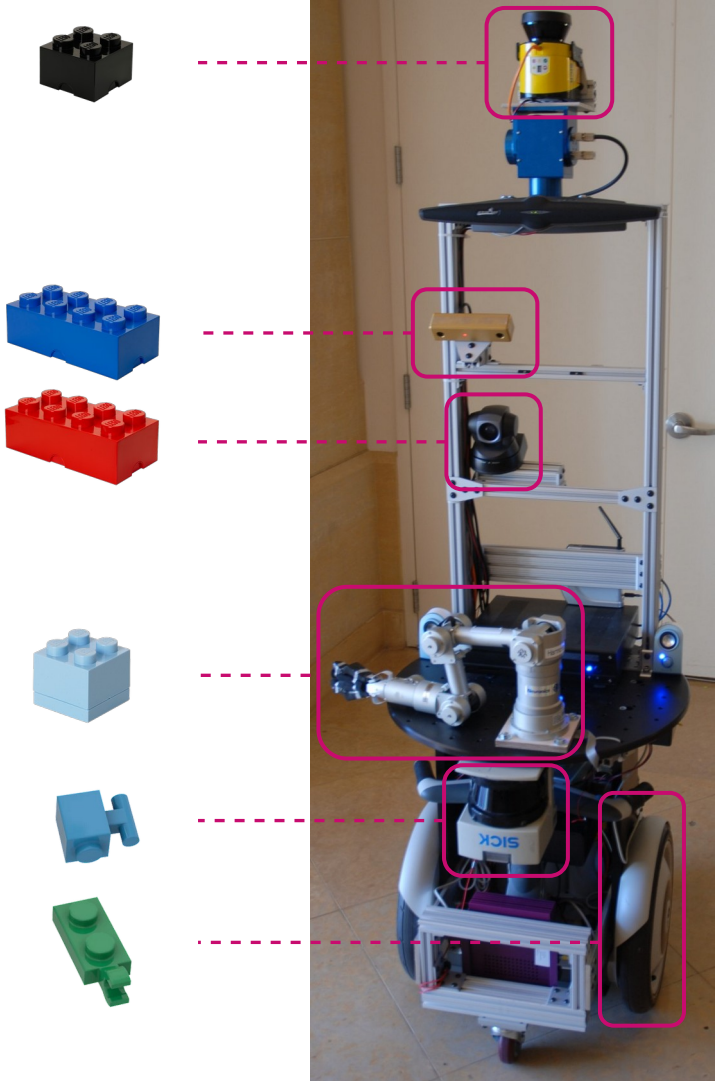
History of ROS

当时，斯坦福人工智能机器人项目涵盖了许多类似的机器人，而不仅仅是单一型号。这使得摩根的代码维护任务异常困难。每种不同型号的机器人都运行独立的程序。

这促使摩根尝试开发一个模块化、分布式的框架，以减轻他的工作量。

History of ROS

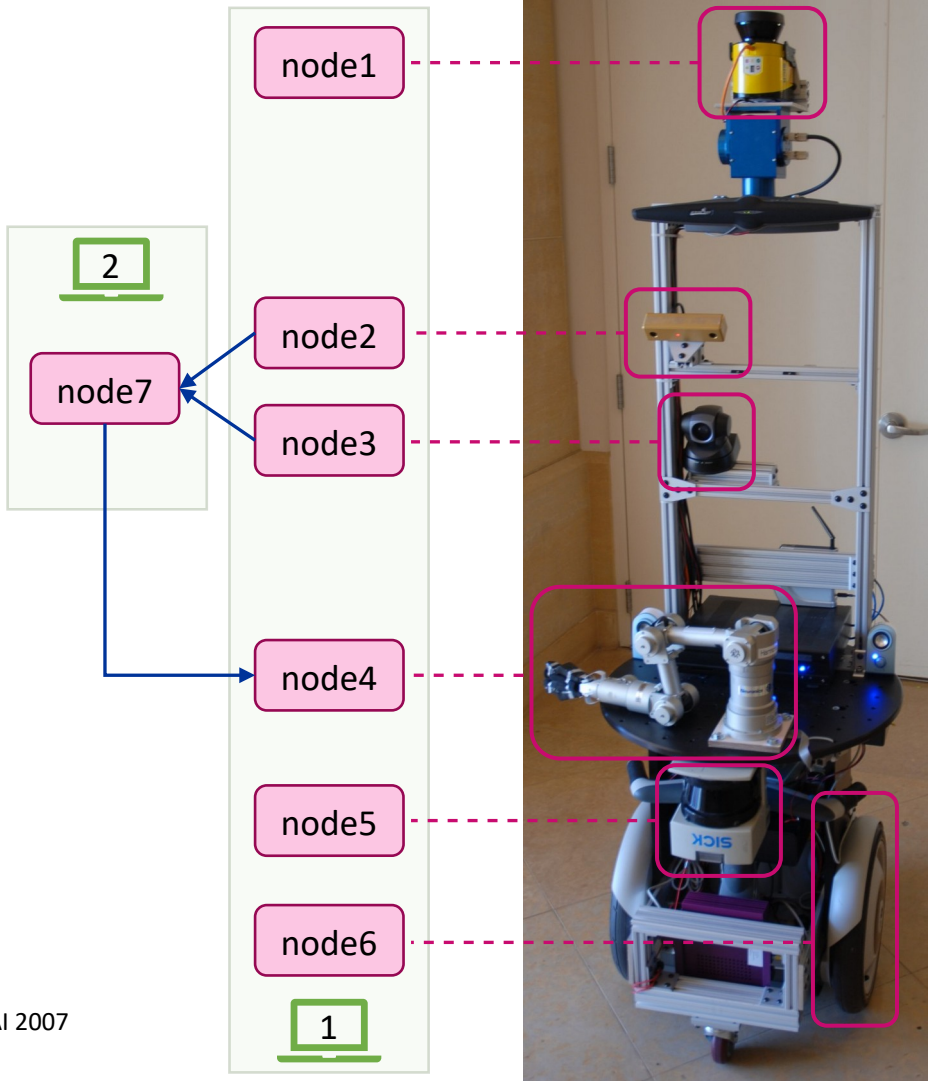
在ROS中，一个完整的机器人程序被分解为独立的模块（称为节点），每个传感器对应一个特定的节点。此外，每个执行器甚至每个算法都可以作为独立的节点存在。在构建机器人时，只需像组装乐高积木一样将这些节点组合在一起。



History of ROS

该程序不仅被划分为独立的节点，而且节点还可以在不同的计算机上运行。此外，每个节点可以使用不同的编程语言开发（通常是 C++ 或 Python）。

这使得控制机器人的模块可以在较小、性能较低的计算机上运行（例如 Raspberry Pi），而处理机器人视觉等任务的节点可以在更强大的计算机上运行。



Quigley et al, “Stair: Hardware and software architecture”, AAAI 2007

History of ROS

随后，Willow Garage 公司与 Ng 合作开发了一个机器人项目，Morgan 担任首席研究员。在该项目中，他完善了 ROS 的概念，并将 ROS 的第一个版本（ROS 1.0——Box Turtle）应用于公司的第一款机器人产品 PR2。

Box Turtle

PR2（2010）

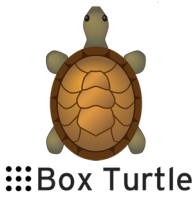
Quigley et al "ROS: an open-source Robot Operating System." ICRA 2009.

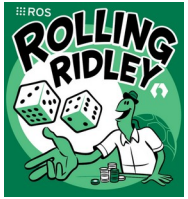
History of ROS

Open Source Robotics Foundation

2012年，摩根获得了博士学位，并创立了ROS基金会（ROSF），致力于ROS的开发和维护。我们今天使用的各种版本的ROS系统都是由该基金会开发和维护的。

foundation.





1495 UNIVERSITY OF ABERDEEN

History of ROS

ROS 有两个主要版本：ROS 1 和 ROS 2。

ROS 1 专注于提供一个用于机器人软件开发的强大框架，主要为研究和教育目的设计，具有单线程通信模型。它主要针对 Ubuntu。

ROS 2 是一种面向支持实际应用的进化，具有多线程架构、增强的安全性、实时能力以及对多操作系统的支持，满足科研和工业的需求。

课程将介绍 ROS 2。

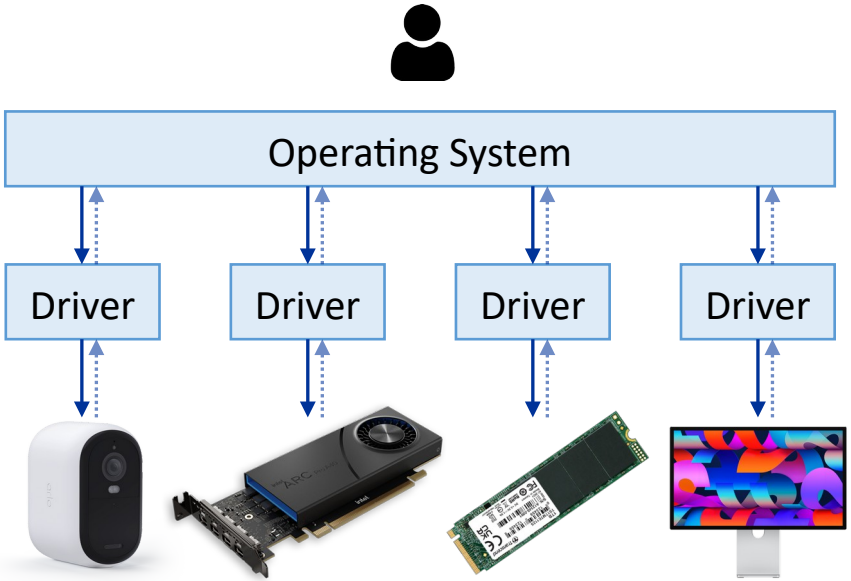
ROS 1

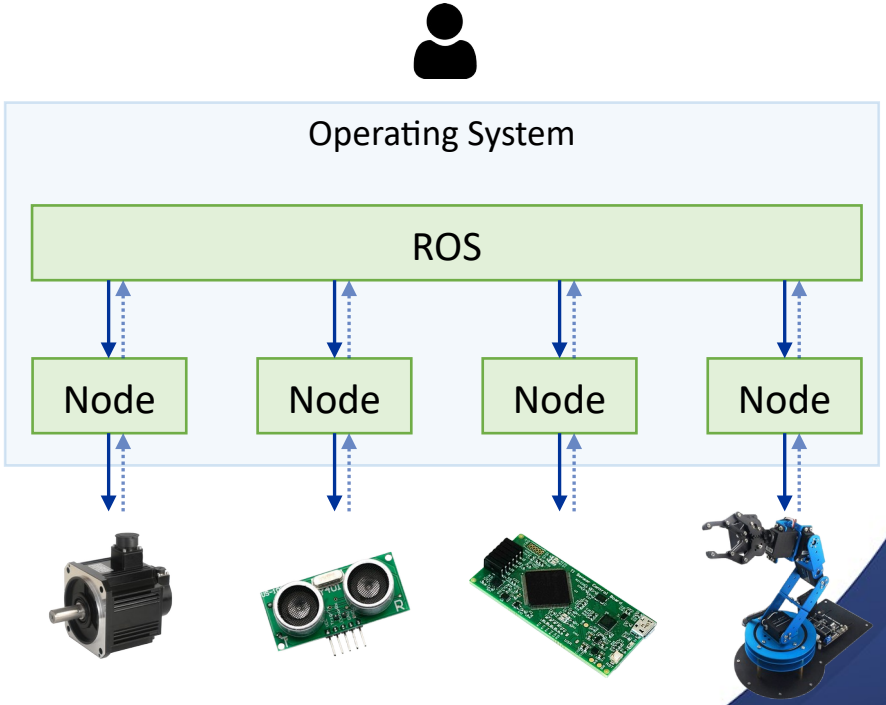
ROS 2

ROS Philosophy

- ROS 系统是围绕点对点架构设计的，由众多紧凑的计算机程序组成，这些程序通过持续交换消息进行通信。
- 该框架是基于工具的，提供一系列小型的通用程序，能够执行各种任务，包括可视化、记录和绘制数据流。
- 它是多语言的，允许使用任何提供客户端库的编程语言开发软件模块，目前支持 C++、Python、LISP、Java、JavaScript、MATLAB、Ruby 等。
- ROS 推崇精简的设计理念，鼓励创建独立的库，然后通过消息将其与其他 ROS 模块进行通信。
- ROS 完全免费且开源，促进了机器人软件开发和创新的协作环境。

ROS vs Operating System





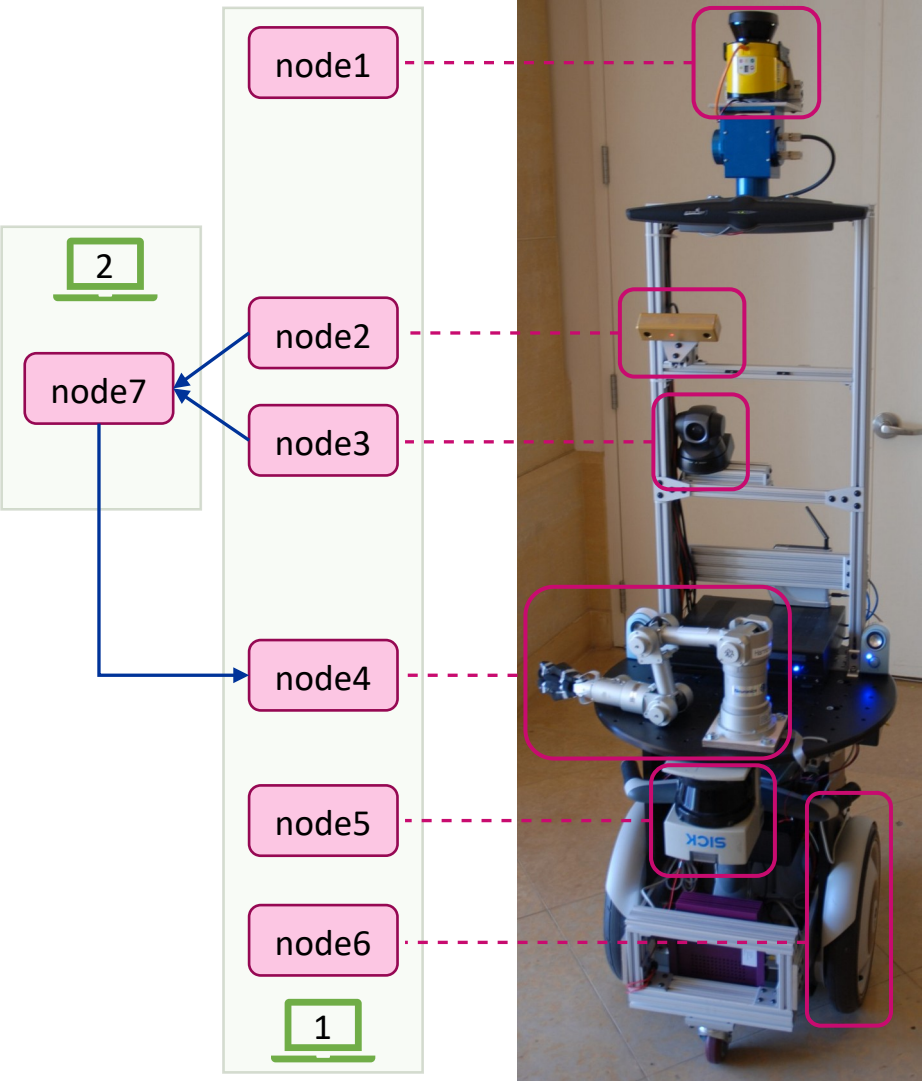
ROS 核心概念

1495 UNIVERSITY OF ABERDEEN

Node

在ROS中，节点表示系统中最小的可执行实体（即程序）。

节点被设计用来执行特定任务，范围从感知和执行到计算和决策过程。在正常情况下，每个节点负责单一功能。具有复杂功能的机器人是由大量具有简单功能的节点组成的。

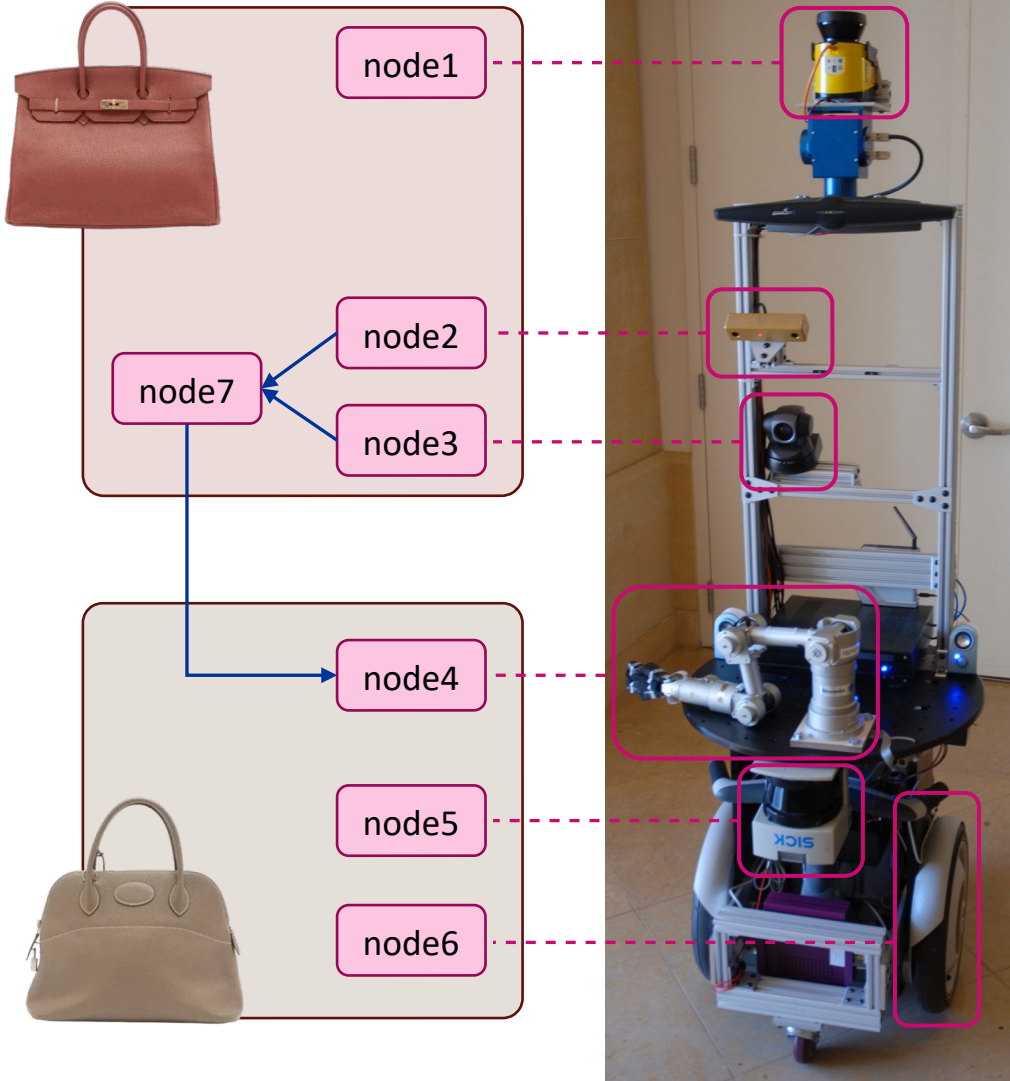


Package

包裹作为基本单位
软件组织，包括由节点、库和其他资源组成的集合，统一用于实现特定目标

在机器人应用中的功能或用途。

ROS 中的包就像 Python 中的库。 当你使用节点时，尤其是当你下载节点时，你需要下载
包含节点的包。



Discovery of Nodes

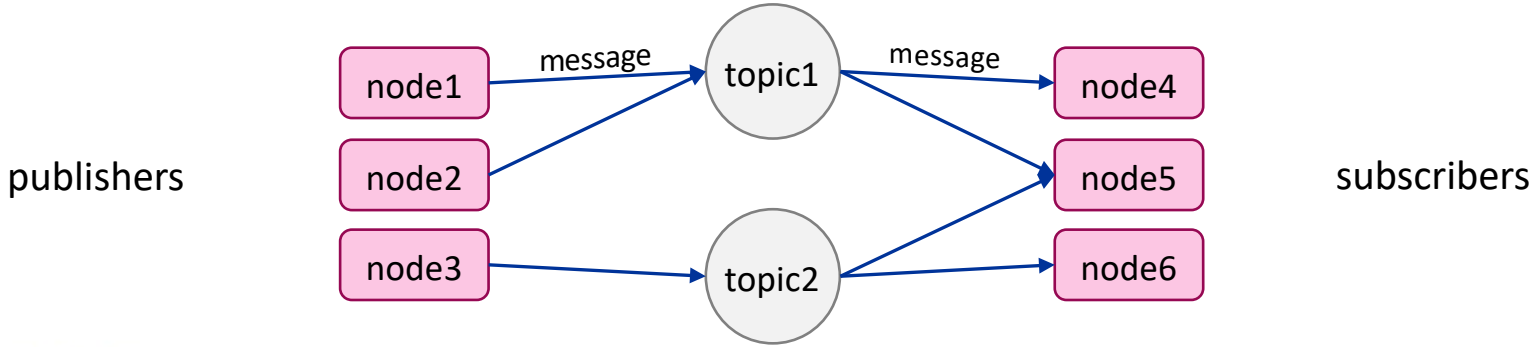
节点的发现由 ROS 自动管理。当一个节点开始或停止运行时，ROS 会将该节点的信息通告给其他活动节点，从而允许它们在节点启动时自动建立数据通道。

节点只有在具有兼容的服务质量设置时才会与其他节点建立连接。

Topic and Message

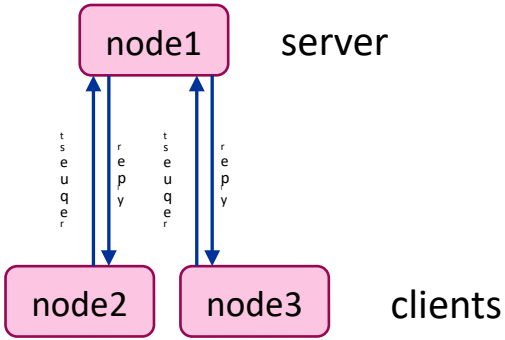
主题是一个命名的通道，节点通过发布/订阅模型（即观察者模式）在其上交换数据，而消息是一种用于节点之间通信的结构化数据类型，定义了通过主题通道传输的信息类型。

注意：主题和消息是 ROS 框架支持的对象。它们不是节点。



Services

服务是一种同步的请求/响应通信机制，其中一个节点（客户端）向另一个节点（服务端）发送请求并等待响应。服务对于需要回复的任务非常有用，例如检索节点状态或执行计算。



Parameters

参数用于存储和管理配置数据。它们使节点能够通过集中式参数服务器在运行时存储和访问配置值。该系统允许节点根据动态信息配置自身和其他节点，从而避免了硬编码值的必要性。

参数也使用观察者模式；节点可以订阅特定参数，当该参数发生变化时，ROS将通过回调机制通知节点。

与 ROS 一起踏上我们的旅程

1495 UNIVERSITY OF ABERDEEN

入门

学习 ROS 和获得支持的主要资源是 ROS 官方网站：ros.org。

首先，点击“入门”以找到为不同目的量身定制的许多资源。

https://ros.org/

Primary ROS Resource

ROS软件包文档

- 核心 ROS 软件包的文档以及特定软件包的内容都托管在 docs.ros.org 上。在该网站上，您可以找到项目的核心教程和文档，以及各个软件包的生成 API 文档。

机器人技术交流

- 如果文档没有解决你的问题，下一个就是 Robotics Stack Exchange。

ROS 指数

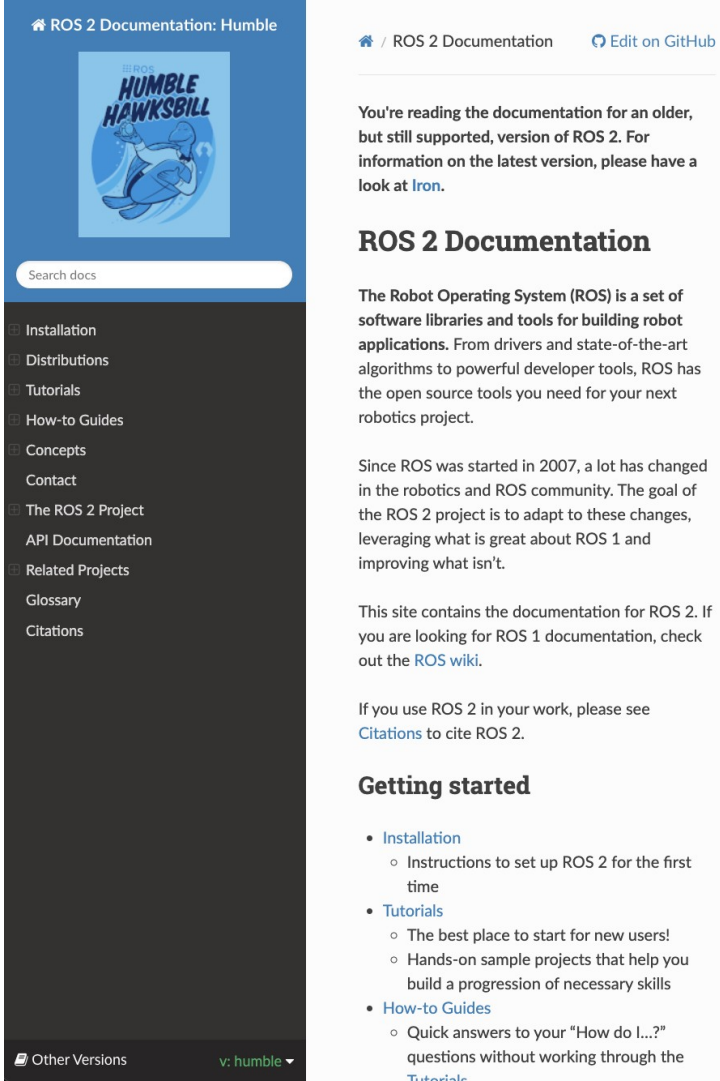
- 当你想要查找有关特定软件包的信息时，索引是最好的起点。它将你连接到与软件包相关的所有重要位置。除了 ROS 软件包的官方文档之外，Wiki 还包含两个你应该参考的关键资源：故障排除指南和常见问题解答。

ROS Package Documentation

ROS 2 文档对初学者非常有帮助，提供了全面的安装说明和教程。

- 为多个系统提供详细的安装指导（主要针对 Ubuntu 和 Windows，而其他系统的标准则更为严格）。

- 这些教程包含大量针对新手的教学内容和示例。对于刚开始学习 ROS 2 的人来说，它们是非常宝贵的资源。



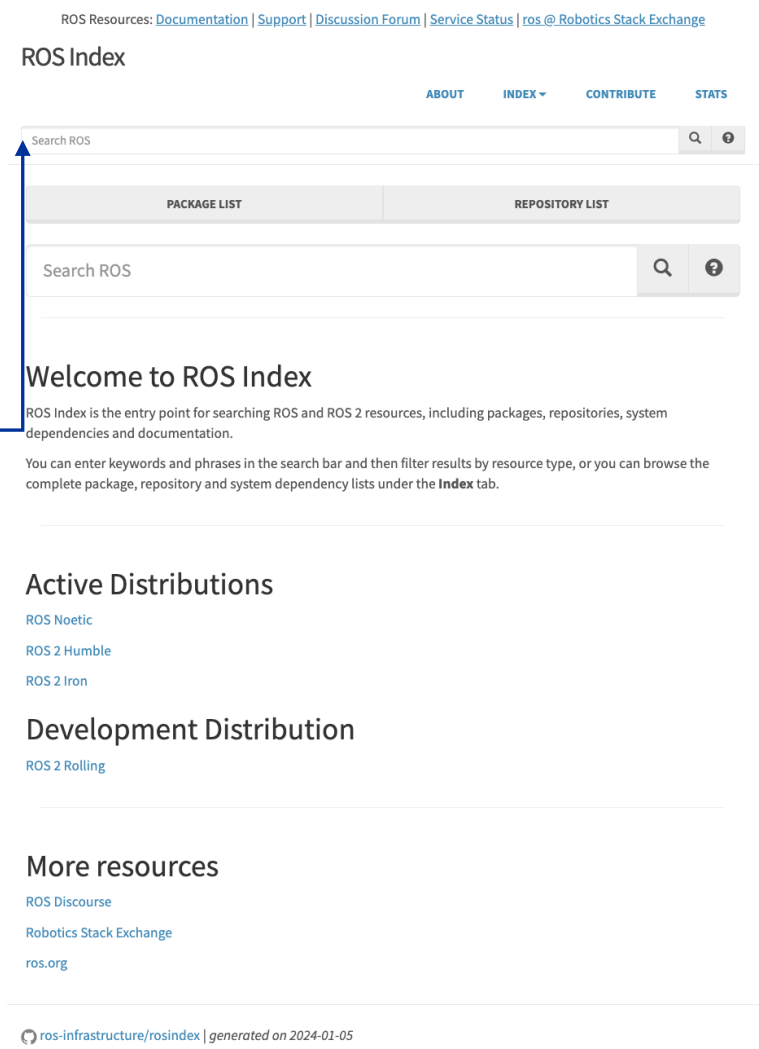
UNIVERSITY OF ABERDEEN

ROS Index

The ROS index contains all downloadable third-party packages and nodes, as well as their descriptions.

You can list the package/node name or search with their names.

E.g. we search ‘turtlesim’ a package name.



UNIVERSITY OF ABERDEEN

ROS Index

搜索结果列在表格中。

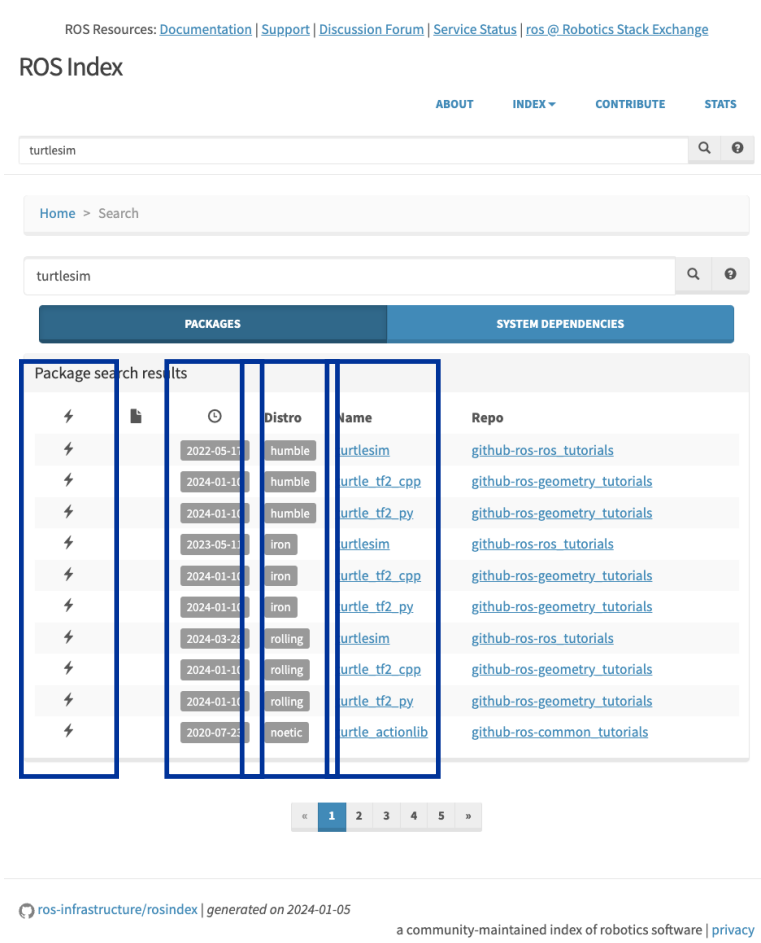
这意味着这些软件包已发布，因此你可以下载它们。

日期是最后更新日期（包括文档更新）。

Distro 指的是目标 ROS 版本。你可以找到一个支持多个 ROS 版本的软件包（但通常不支持所有可能的版本）。

这些名称是包名。

我们并不总是安装最新版本的 ROS，因为很难获得足够的包支持。

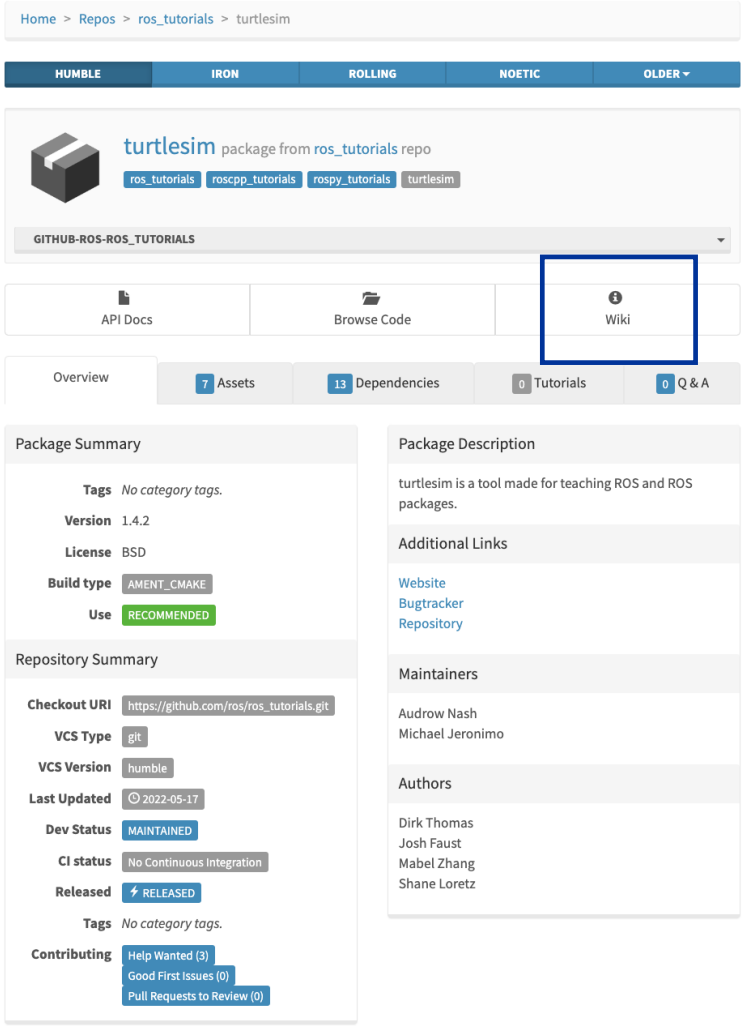


UNIVERSITY OF ABERDEEN

ROS Index

在包页面上，您可以找到包概览、仓库概览以及其他信息。

最重要的是，维基页面包含使用该软件包的具体说明。



1495 UNIVERSITY OF ABERDEEN

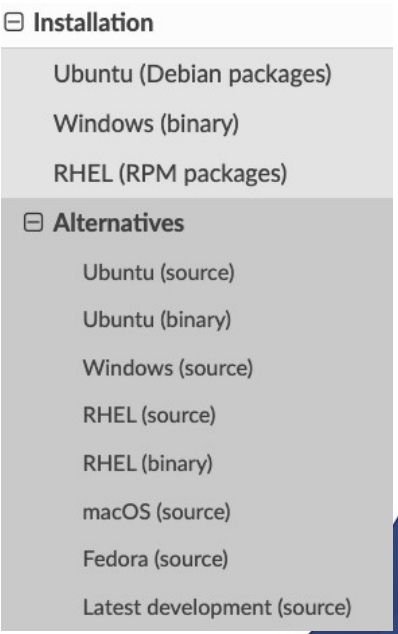
ROS2 安装

详细的安装说明可以在 ROS 软件包文档中找到。

尽管它支持多种操作系统，但主要情况下，人们在 Ubuntu 上运行 ROS。在非 Ubuntu 的操作系统中，你需要安装跨平台支持的应用程序（例如 Qt5）来为 ROS 提供兼容 Ubuntu 的环境。

这里是 ROS2 Humble 的安装说明链接。

- Ubuntu: https://docs.ros.org/en/humble/Installation/Ubuntu-Install-Debians.html
- Windows: https://docs.ros.org/en/humble/Installation/Windows-Install-Binary.html



运行 ROS 2 节点

ROS 操作主要通过 CLI（命令行界面）实现。我们使用随 ROS 提供的小海龟示例来展示如何运行 ROS 节点。

首先我们打开命令行，然后我们需要 ROS 环境 结束

```
source /opt/ros/humble/setup.bash
```

在 Windows 中，使用 PowerShell

```
C:\dev\ros2_humble\local_setup.ps1
```

你需要在每次打开新终端时运行此命令以访问 ROS2。如果你不想每次都这样做，请参见 https://docs.ros.org/en/humble/Tutorials/Beginner-CLI-Tools/Configuring-ROS2-Environment.html。

运行 ROS 2 节点

在 Ubuntu 中，我们使用以下命令安装 turtlesim 包：

```
sudo apt install ros-humble-turtlesim
```

ros version

package name

在 Windows 中，ROS 集成了一些常用的软件包，包括 turtlesim，因此我们可以跳过安装步骤。

然而，如果你想使用一个没有与 ROS 集成的包，就会变得非常复杂。你需要下载该包的源代码并在本地进行编译。

运行 ROS 2 节点

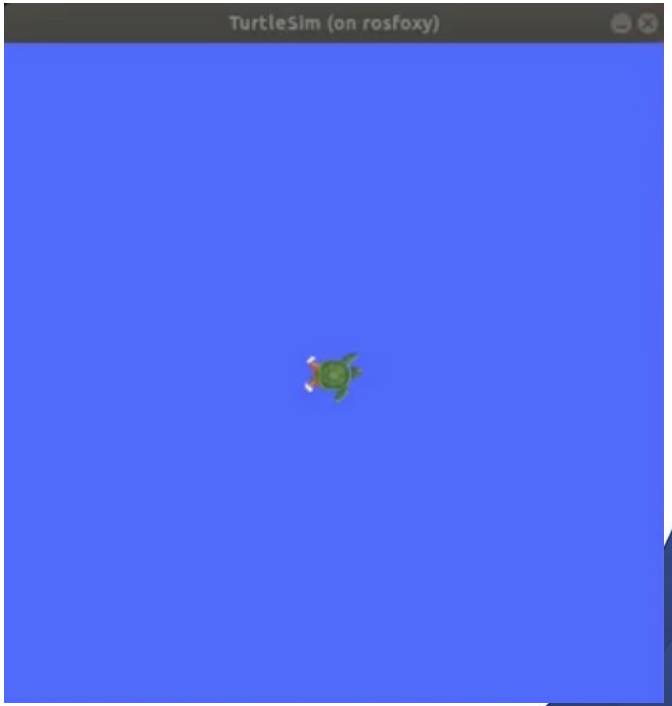
然后，要启动 turtlesim，请在终端中输入以下命令：

```
ros2 run turtlesim turtlesim_node
```

package name

node name

你将启动正确的窗口。

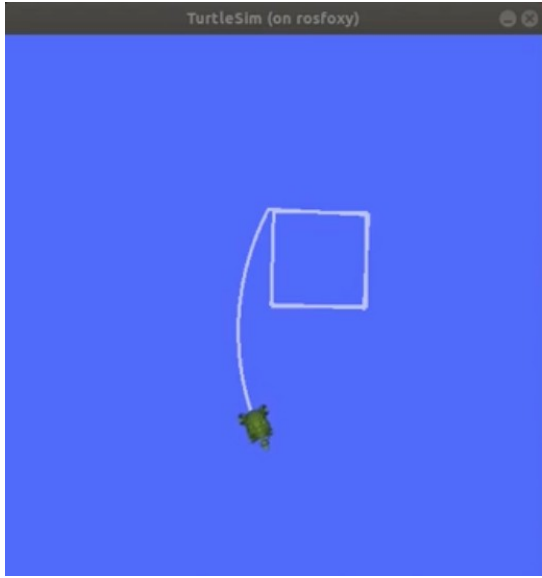


运行 ROS 2 节点

ROS 支持多节点协作。为了控制这只小海龟，我们需要运行另一个控制节点。为此，我们需要再次打开一个新的命令行窗口，然后你将运行一个新节点来控制第一个节点中的海龟：

ros2 运行 turtlesim turtle_teleop_key

此时，你应该打开三个窗口：一个运行 turtlesim_node 的终端、一个运行 turtle_teleop_key 的终端，以及 turtlesim 窗口。安排这些窗口，使你可以看到 turtlesim 窗口，同时保持运行 turtle_teleop_key 的终端处于活动状态，以便你可以在 turtlesim 中控制乌龟。




```
ubuntu@rosfoxy:~$ ros2 run turtlesim turtlesim_node
QStandardPaths: XDG_RUNTIME_DIR not set, defaulting to '/tmp/ru
[INFO] [1591730274.971844433] [turtlesim]: Starting turtlesim w
rtlesim
[INFO] [1591730274.977795804] [turtlesim]: Spawning turtle [tur
445], y=[5.544445], theta=[0.000000]
```

```
ubuntu@rosfoxy:~$ ros2 run turtlesim turtle_teleop_key
Reading from keyboard
---------------------------
Use arrow keys to move the turtle.
Use G|B|V|C|D|E|R|T keys to rotate to absolute orientations. 'F
ation.
'Q' to quit.
```

开发你的第一个节点

1495
UNIVERSITY OF
ABERDEEN

Create Workspace Folder

工作空间是包含 ROS 2 软件包的文件夹。

单个工作区可以包含任意多的包，每个包都有自己的文件夹。你还可以在一个工作区中使用不同构建类型的包（CMake、Python 等）。你不能有嵌套的包。

最佳做法是在你的工作区内有一个 src 文件夹，并在其中创建你的包。这可以保持工作区的顶层“干净”。

所以，我们的第一步是简单地创建工作区文件夹。

一个简单的工作区可能看起来像：

```
workspace_folder/
    src/
      cpp_package_1/
          CMakeLists.txt
          include/cpp_package_1/
          package.xml
          src/

      py_package_1/
          package.xml
          resource/py_package_1
          setup.cfg
          setup.py
          py_package_1/
      ...
      cpp_package_n/
          CMakeLists.txt
          include/cpp_package_n/
          package.xml
          src/
```

Create A Package

确保你在你的_workspace/src/ 文件夹中。

然后，运行打包创建命令:

```
ros2 pkg create --build-type ament_python py_hello_world
```

use python

package name

Then, it should generate a folder like:

```
py_hello_world/
      package.xml
      resource/py_hello_world
      setup.cfg
      setup.py
      py_hello_world/
```

Create Node File

在 py_hello_world/py_hello_world 中，创建一个文件 hello_world_node.py。

然后，经过代码。

```python
import rclpy
from rclpy.node import Node

class HelloWorldNode(Node):
    def __init__(self):
        super().__init__('hello_world_node')
        self.timer = self.create_timer(2, self.publish_hello_world)
        self.get_logger().info('Hello World Node has been started.')

    def publish_hello_world(self):
        self.get_logger().info('Hello, world!')

def main(args=None):
    rclpy.init(args=args)
    node = HelloWorldNode()
    try:
        rclpy.spin(node)
    except KeyboardInterrupt:
        pass
    finally:
        node.destroy_node()
        rclpy.shutdown()

if __name__ == '__main__':
    main()
```

Update Setting

在 py_hello_world/setup.py 的 entry_points 条目中添加以下设置

```
entry_points={
'console_scripts': [        'hello_world = py_hello_world.hello_world_node:main',     ],



},
```

Build The Node

回到你的工作区文件夹：your_workspace/ 现在你可以构建你的包了：

colcon 构建

注意：此命令会构建你工作空间中的所有内容。

Run The Node

要使用您的新软件包和可执行文件，首先打开一个新的终端并加载您的主要 ROS 2 安装。

```
source install/local_setup.bash
```

要运行您在创建软件包时使用 --node-name 参数创建的可执行文件，请输入以下命令：

```
ros2 运行 py_hello_world hello_world
```

结果如下：

```
[INFO] [hello_world_node]: Hello World Node has been started.
[INFO] [hello_world_node]: Hello, world!
[INFO] [hello_world_node]: Hello, world!
```

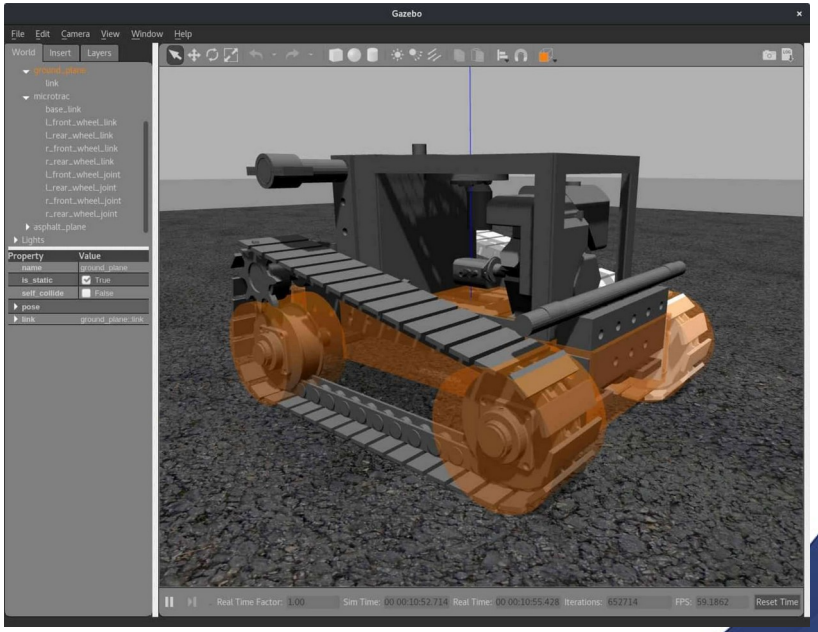
Gazebo

凉亭

Gazebo 是一款先进的机器人仿真软件，它能够在复杂的室内和室外环境中进行机器人的设计和测试。

学习资源：

- https://gazebosim.org/
- https://gazebosim.org/docs/all/getstarted
- https://docs.ros.org/en/humble/Tutorials/Advanced/Simulators/Gazebo/Gazebo.html



UNIVERSITY OF ABERDEEN

凉亭

<Video: 02 - gazebo overview.mp4>

结论

我们介绍了ROS的核心概念，包括节点、主题、服务、消息传递和软件包，这些都是构建复杂机器人系统所必需的。

通过一个简单的例子，我们演示了如何创建和运行一个 ROS 节点，这是学习 ROS 的第一步。

最后，ROS不仅仅是技术；它作为学术界与工业之间的桥梁，促进了机器人技术的快速发展和应用。